

MuJoCo + VLA

토이프로젝트

3조

박소영, 선주영, 유혜민, 최동석

Contents

01 프로젝트 개요

02 코드 설명

03 시연

01 프로젝트 개요

LiDAR 센서의 sensordata로 거리 측정

- 카드와 로봇 사이 실제 거리 (좌표값으로 계산)과 LiDAR sensordata로 측정된 거리 비교
- 둘 사이 오차가 나는 이유에 대해 학습

화살표 방향 카드를 보고 그에 맞는 동작을 수행하도록 수정

- 카드의 이미지를 변경하여 해당 객체를 인식할 수 있도록 다시 학습
- 카드 객체와의 거리를 lidar로 측정하여 일정 거리 이하가 되면 해당 화살표가 가리키는 방향과 동일하게 동작

02 코드 설명-1

scripts/tb3_sim.py

👉 `get_target_distance_and_time`: Lidar의 센서 값을 기반으로 카드와의 거리와 카드까지 도달하는 데 걸리는 시간 (속도는 `turtlebot3`의 최대 속도인 `0.22m/s`라 가정하고 측정)을 계산

```
def get_target_distance_and_time(self, target_label: str):
    # YOLO 결과 가져오기
    det_dict = self.yolo_detect_dict()
    if target_label not in det_dict:
        return None, None

    target_obj = det_dict[target_label][0]
    cx, cy = target_obj["center"]
    x1, y1, x2, y2 = target_obj["bbox"]
    bbox_height = y2 - y1

    # 1. LiDAR 데이터 가져오기 (사용자 XML의 'laser' 이름 기준 우선 시도)
    ranges = np.array([])
    # 센서 이름 우선순위: XML 구성에 따라 'laser', 'rf', 'scan' 순서
    for name in ["laser", "rf", "scan", "lidar"]:
        temp_ranges = Lidar.get_lidar_ranges(self.model, self.data, name)
        if len(temp_ranges) > 0:
            ranges = temp_ranges
            break
```

```
# 2. 타겟 거리 추출 로직 (get_target_distance_and_time 내부 수정)
if len(ranges) >= 360:
    # 1. 인덱스 계산 (정밀도 유지를 위해 round 사용)
    target_angle_deg = (cx - 320) / 640 * 60.0
    lidar_idx = int(round(target_angle_deg)) % 360

    # 2. 검색 범위 확대 (가장자리 카드를 위해 +-8도까지 탐색)
    search_range = 8
    indices = [(lidar_idx + i) % 360 for i in range(-search_range, search_range + 1)]

    # 3. 유효 거리 필터링 (0.2m ~ 3.5m 사이만 '카드'로 인정)
    # 0.2m 미만: 로봇 몸체에 반사된 노이즈 방지
    # 3.5m 초과: 저 멀리 있는 벽 데이터 방지
    valid_samples = [ranges[i] for i in indices if 0.2 < ranges[i] < 3.5]

    if valid_samples:
        # 카드 후보 중 가장 가까운 거리를 선택
        final_dist = np.min(valid_samples)
    else:
        # LiDAR가 놓쳤을 경우 Vision Fallback (bbox 높이 기반)
        if bbox_height > 0:
            final_dist = 160.0 / bbox_height
        else:
            final_dist = 15.0
```

```
# 3. Vision Fallback 및 시간 계산 (기존 로직 유지)
if final_dist >= 14.5 and bbox_height > 0:
    final_dist = 160.0 / bbox_height # Fallback

target_gap = 0.10
robot_speed = 0.22
time_to_stop = max(0.0, (final_dist - target_gap) / robot_speed)

return final_dist, time_to_stop
```

02 코드 설명-1

scripts/tb3_sim.py

👉 `get_ground_truth_distance`: 카드와 로봇 body 사이의 물리적 거리(카드의 geom 위치와 로봇의 body 위치 차이)를 계산

👉 LiDAR로 측정한 거리와 ground truth의 출력을 비교하며 LiDAR 센서 출력에 대해 학습

```
def get_ground_truth_distance(self, target_label: str):
    # MuJoCo 시뮬레이션 내부 좌표를 사용하여 '실제 물리적 거리'를 계산
    try:
        # 1. 타겟 Body의 위치 가져오기
        body_name = XML_BODY_MAP.get(target_label, target_label)
        target_id = mj.mj_name2id(self.model, mj.mjtObj.mjOBJ_BODY, body_name)
        if target_id == -1: return None
        target_pos = self.data.xpos[target_id]

        # 2. 로봇의 기준점 위치 가져오기
        try:
            robot_id = mj.mj_name2id(self.model, mj.mjtObj.mjOBJ_BODY, "base")
            robot_pos = self.data.xpos[robot_id]
        except:
            return None

        # 3. 2D 평면 거리(x, y) 계산
        dist = np.linalg.norm(target_pos[:2] - robot_pos[:2])
        return dist
    except:
        return None
```


02 코드 설명-2 scripts/prompt.yaml

👉 거리를 측정하기 위한 규칙들을 상단에, 거리 및 카드까지 도달하는 데 걸리는 시간을 예시 질문의 [Observation]에 추가

```
[Question]
우회전 카드까지 얼마나 남았어?

[Observation]
{
  "turn_right": [
    {
      "confidence": 0.9,
      "bbox": [ ... ],
      "center": [320, 240],
      "distance_m": 1.5,
      "time_to_stop_10cm_sec": 6.36
    }
  ]
}

[Answer]
Action: 직진
Response: 우회전 카드까지 약 1.5미터 남았어요. 10cm 앞까지 가려면 약 6.4초 정도 걸릴 것 같아서 계속 직진할게요.
```

3. Observation은 JSON 형식일 수 있습니다. 각 객체의 bbox와 center외에도 'distance_m'(현재 거리)와 'time_to_stop_10cm'(도착 예상 시간) 정보가 포함되어 있습니다.
- 사용자가 거리나 시간을 물어보면, 절대 추측하지 말고 JSON에 있는 'distance_m'과 'time_to_stop_10cm' 수치를 그대로 읽어서 답하세요.

거리 관련 추가적인 규칙입니다.

18. 사용자가 "얼마나 멀어?", "거리가 어떻게 돼?"라고 물으면 'distance_m' 값을 참고하여 "약 0.00미터 떨어져 있어요"라고 답변하세요.
19. 사용자가 "가까이 가는데 얼마나 걸려?", "도착하려면 몇 초 걸려?"라고 물으면 'time_to_stop_10cm_sec' 값을 참고하여 답변하세요.
20. 만약 'distance_m' 값이 0.15 이하라면, Action은 '멈춤'을 선택하고 "충분히 가까이 왔어요"라고 답변하세요.
21. 각 객체에는 'bbox', 'center' 외에도 'distance_m'(거리)와 'time_to_stop_10cm_sec'(10cm 앞 정지까지 남은 시간) 정보가 포함될 수 있습니다.

02 코드 설명-3

scripts/gemini_tb3.py

👉 감지한 객체들에 대한 거리 정보 반환하는
코드를 talk 메서드 안에 추가

```
# --- [수정] 감지된 객체들에 대해 거리 정보 추가 ---  
# det_dict 구조: {'heart': [{'bbox':..., 'center':...}], ...}  
  
for label, objects in det_dict.items():  
    # 시뮬레이터에게 해당 라벨까지의 거리와 시간 물어봄  
    real_dist, real_time = sim.get_target_distance_and_time(label)  
  
    # 값이 유효하다면 리스트 안의 모든 객체(또는 첫번째)에 정보 주입  
    if real_dist is not None:  
        for obj in objects:  
            obj['distance_m'] = round(real_dist, 2)          # 미터 단위 (소수점 2자리)  
            obj['time_to_stop_10cm'] = round(real_time, 2)  # 초 단위  
  
# =====
```

02 코드 설명-4

utils/mujoco_renderer.py

scripts/tb3_sim.py (render)메서드

👉 main 창에 추가적인 텍스트를 Orientation, Gyro, Acceleration과 함께 우측 상단에 표시하기 위한 코드를 추가

👉 하단에 lidar로 측정한 거리 및 월드 상의 실제 거리와의 오차(%)를 추가하여 lidar 성능을 평가 가능하게 수정

👉 lidar의 경우 객체가 frame의 외곽 쪽에 있을 수록 측정한 거리의 오차가 커짐

```
if self.render_counter % 10 == 0 or self.cached_gt_info == "":
    gt_info_str = "Distance to Cards (GT) | Error:\n"
    for label in ["go_straight", "rotate", "turn_left", "turn_right"]:
        res = self.get_target_distance_and_time(label)
        dist = res[0] if res is not None else None
        real_dist = self.get_ground_truth_distance(label)

        error_rate = None
        if dist is not None and real_dist is not None and real_dist != 0:
            error_rate = abs(dist - real_dist) / real_dist * 100.0

        if real_dist is not None:
            dist_str = f"{dist:.2f}m" if dist is not None else "N/A"
            err_str = f"({error_rate:.1f}%)" if error_rate is not None else "(N/A)"
            gt_info_str += f" {label:<12}: {dist_str:>7} {err_str:>8}\n"
        else:
            gt_info_str += f" {label:<12}: N/A\n"
```


02 코드 설명-5

asset/robotics_tb3/factory_card.xml

👉 변경된 카드 이름들에 맞게 factory_cards.xml
파일의 body 이름들을 수정

```
<!-- 카드들도 z=0 근처에 놓혀 놓기 -->
<!-- euler="0 0 0" : 회전 없이 기본 방향 -->

<body name="card_go_straight" pos="-1.5 -2 0.3" euler="-1.57 0 -1.57">
  <geom class="card" material="ace_of_clubs_mat"/>
  <geom type="box" size="0.047 0.032 0.0005" mass="0" group="3" friction=".2"/>
</body>

<body name="card_rotate" pos="-0.7 -2 0.3" euler="-1.57 0 -1.57">
  <geom class="card" material="ace_of_diamonds_mat"/>
  <geom type="box" size="0.047 0.032 0.0005" mass="0" group="3" friction=".2"/>
</body>

<body name="card_turn_right" pos="0.7 -2 0.3" euler="-1.57 0 -1.57">
  <geom class="card" material="ace_of_hearts_mat"/>
  <geom type="box" size="0.047 0.032 0.0005" mass="0" group="3" friction=".2"/>
</body>

<body name="card_turn_left" pos="1.5 -2 0.3" euler="-1.57 0 -1.57">
  <geom class="card" material="ace_of_spades_mat"/>
  <geom type="box" size="0.047 0.032 0.0005" mass="0" group="3" friction=".2"/>
</body>
```

02 코드 설명-6

asset/robotics_tb3/tb3_burger_sensor.xml

👉 rangefinder(Lidar) sensor의 cutoff
3.0 -> 15.0 으로 수정

```
<sensor>
  <!-- LiDAR (cutoff 변경)-->
  <rangefinder name="laser" site="rf" cutoff="15.0"/>
```

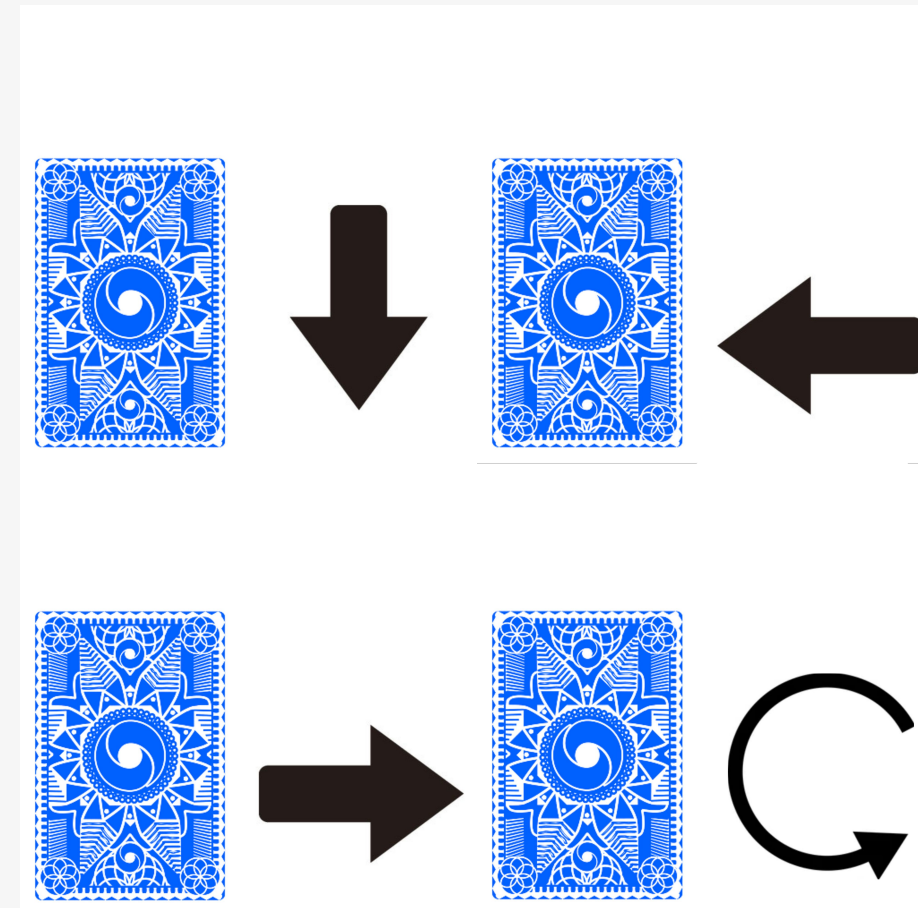
02 코드 설명-7

asset/robotis_tb3/cards

👉 cards 디렉토리에 go_straight.png, rotate.png, turn_left.png, turn_right.png를 추가

👉 기존에 트럼프 카드를 인식 하던 것을 화살표 사진으로 변경하여 인식할 객체를 변경

👉 인식할 객체가 변경됨에 따라 roboflow에서 다시 학습을 시켜 얻은 [best.pt](#) 가중치 파일을 저장



시연

감사합니다.